Apuntes · Clase 45 · Redes neuronales

Para leer antes de los ejercicios · lectura de ~40 min · pensado para quien mira por primera vez dentro de una red neuronal

Lo más importante, en 8 ideas

1. **Una neurona.** es una suma ponderada más un sesgo, pasada por una función de activación.
2. **Una capa.** son muchas neuronas a la vez: X·W + b y la activación.
3. **La activación no lineal.** es lo que impide que varias capas se reduzcan a una sola.
4. **Softmax y −log p.** convierten puntajes en probabilidades y miden el error de una imagen.
5. **Aprender es minimizar J.** con descenso de gradiente: $w^{+} = w - \eta \cdot J'(w)$.
6. **Retropropagación.** calcula los gradientes de todos los pesos con la regla de la cadena.
7. **Lotes, épocas y Adam.** organizan el entrenamiento; el optimizador decide el paso.
8. **Ancho, alpha y épocas.** son hiperparámetros: se eligen con validación; la prueba se mira al final.

Cada sección trae una explicación en palabras simples, un **ejemplo resuelto con números distintos** a los de las diapositivas y los ejercicios, y un resumen «para recordar». Ejercicios en `C45_Ejercicios_Cuaderno_RedesNeuronales.pdf` y notebook `C45_RedesNeuronales.ipynb`.

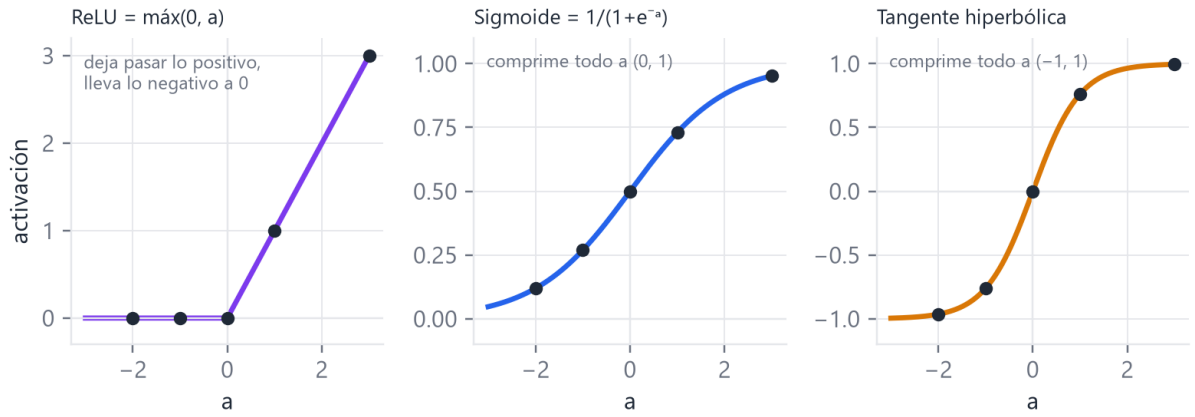
1 La neurona: suma ponderada, sesgo y activación

Una **neurona artificial** es una calculadora diminuta con tres piezas: unos **pesos** (uno por cada entrada), un **sesgo** (un número que se suma al final) y una **función de activación**. Primero multiplica cada entrada por su peso y suma todo: $a = w_1x_1 + \ldots + w_px_p + b$. Luego aplica la función: $h = \varphi(a)$. Ese h es la **activación**. Los pesos y el sesgo son los **parámetros**: se ajustan al entrenar. Los x son datos y no cambian. Un peso positivo hace que la entrada empuje la suma hacia arriba; uno negativo, hacia abajo; el sesgo permite que la neurona responda aunque todas las entradas valgan 0.

Ejemplo resuelto · una neurona con 3 entradas

Paso	Cálculo	Resultado
Productos	0.5·2; 1·(−1); (−0.2)·3	1; −1; −0.6
Suma + sesgo	1 − 1 − 0.6 + 0.1	a = -0.5
ReLU	máx(0, −0.5)	h = 0
Sigmoide	1 ÷ (1 + e^0.5)	0.3775
Tangente hiperbólica	tanh(−0.5)	-0.4621

Lectura: la misma suma a = −0.5 da resultados distintos según la función. ReLU la convierte en 0; la sigmoide la deja en (0, 1); tanh, en (−1, 1). **Error común**: pensar que h = 0 significa que la neurona «no hizo nada»: calculó un valor y ese valor es 0.



Lo que debes recordar

- $a = \Sigma\, w_j x_j + b$; $h = \varphi(a)$
- Pesos y sesgo = parámetros que se aprenden; x = datos
- ReLU(a) = máx(0, a); sigmoide en (0, 1); tanh en (−1, 1)

2 Una capa: muchas neuronas y un producto de matrices

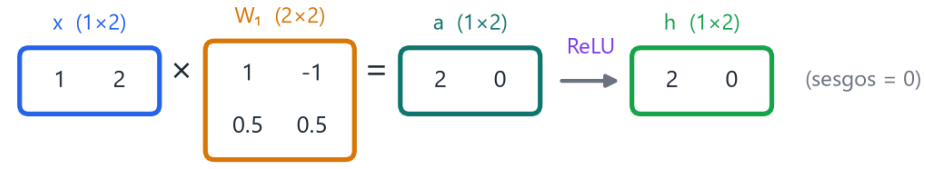
Una **capa** reúne varias neuronas que miran la misma entrada, cada una con sus propios pesos y sesgo. En vez de calcular una por una, se ordenan los pesos en una matriz **W** con una **columna por neurona** y se multiplica: A = X·W + b. Cada **fila de X es una observación** (una imagen), así que una sola multiplicación procesa muchas imágenes. Las dimensiones deben encajar: (n × p) · (p × q) = (n × q), donde n son las observaciones, p las entradas y q las unidades. Se mantiene siempre la misma convención (fila por observación), porque cambiarla a mitad de camino daría resultados inconsistentes.

Ejemplo resuelto · 2 observaciones, 3 entradas, 2 unidades

Paso	Fila 1: x = (2, 1, 1)	Fila 2: x = (0, 3, 1)
Unidad 1: pesos (0.5, 1, −0.5), b = 0.1	1 + 1 − 0.5 + 0.1 = 1.6	0 + 3 − 0.5 + 0.1 = 2.6
Unidad 2: pesos (−1, 2, 0.3), b = 0.2	−2 + 2 + 0.3 + 0.2 = 0.5	0 + 6 + 0.3 + 0.2 = 6.5
ReLU	h = (1.6, 0.5)	h = (2.6, 6.5)

Lectura: X es 2×3, W es 3×2 y el resultado A es 2×2 (una fila por imagen, una columna por unidad). Con 64 píxeles y 32 unidades sería X(360×64)·W(64×32) = A(360×32). **Error común**: olvidar que el sesgo se suma a **todas** las filas.

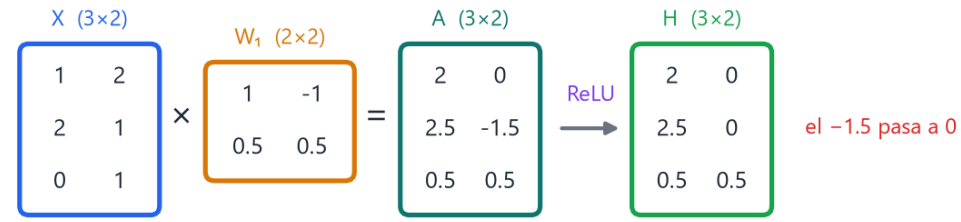
Una observación



cada columna de W_1 = los pesos de una unidad

$a_1 = 1\cdot1 + 2\cdot0.5 = 2$ · $a_2 = 1\cdot(-1) + 2\cdot0.5 = 0$ → h = (2, 0)

Tres observaciones a la vez (cada fila usa los mismos pesos y sesgos)



¿Y por qué se necesita la activación? Porque sin ella, dos capas seguidas se pueden juntar en una: $(xW_1 + b_1)W_2 + b_2 = x(W_1W_2) + (b_1W_2 + b_2)$, que es otra función lineal. En el ejemplo de la guía, con $x = (3, -1)$, dos capas lineales y la capa única equivalente dan el mismo 13.5. La activación **no lineal** rompe esa equivalencia: es la que permite que una red aprenda formas que ninguna recta captura, como separar los puntos de la diapositiva 11.

Lo que debes recordar

- Una columna de W = una neurona; una fila de X = una observación
- $(n\times p)\cdot(p\times q) = (n\times q)$; el sesgo se suma a todas las filas
- Sin activación no lineal, varias capas equivalen a una

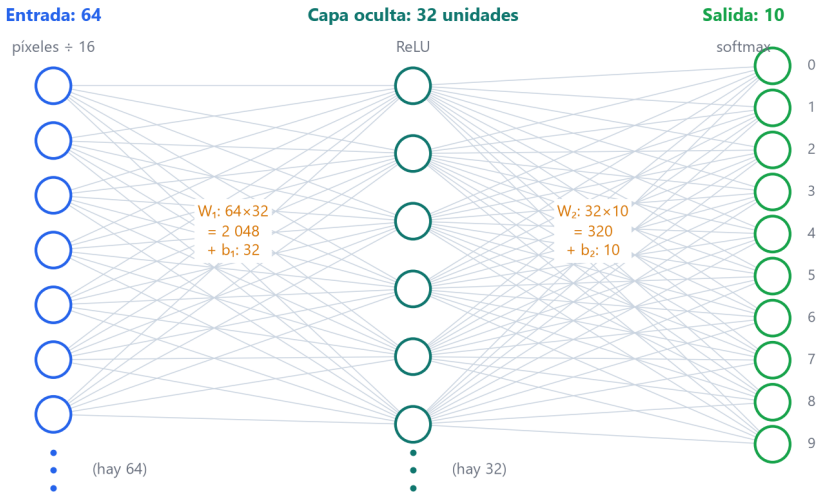
3 Arquitectura, hiperparámetros y conteo de parámetros

La **arquitectura** describe cuántas capas hay y cuántas neuronas tiene cada una, leyendo de izquierda a derecha. «64–32–10» significa: 64 entradas, una capa oculta de 32 unidades y 10 salidas. El número de unidades es el **ancho**; el de capas, la **profundidad**. Estos números son **hiperparámetros**: hay que fijarlos *antes* de ajustar con los datos y no existe una regla que dé el valor óptimo; se prueban. No se confunden con los **parámetros** (pesos y sesgos), que sí se ajustan. Contar parámetros es fácil: cada unidad tiene (número de entradas + 1) parámetros, donde el +1 es el sesgo.

Ejemplo resuelto · una red 8–5–3

Capa	Cálculo	Parámetros
Entrada → oculta (5 unidades)	$5 \cdot (8 + 1) = 5 \cdot 9$	45
Oculta → salida (3 unidades)	$3 \cdot (5 + 1) = 3 \cdot 6$	18
Total	45 + 18	63

Lectura: ocurre lo mismo que con 64–32–10 (2 410 parámetros): una capa más ancha o una capa adicional aumenta muchísimo el número de cosas por ajustar, y con ello el tiempo y el riesgo de sobreajuste. **Error común**: contar las 64 entradas o las 32 activaciones como parámetros; son valores que se calculan, no se aprenden.



Parámetros: 2 048 + 32 + 320 + 10 = 2 410 (con 16 unidades: 64·16+16 + 16·10+10 = 1 210)

Lo que debes recordar

- Parámetros: pesos y sesgos (se aprenden). Hiperparámetros: capas, ancho, tasa, lote, alpha, épocas (se fijan antes)
- Por unidad: (entradas + 1) parámetros
- 64–32–10 tiene 2 410; 64–16–10, 1 210

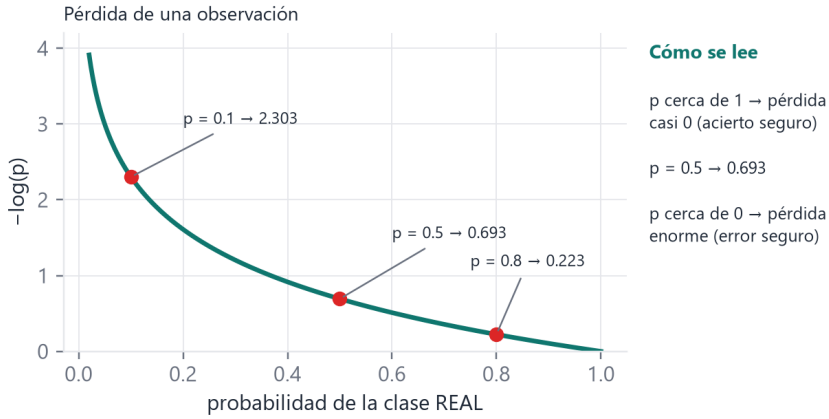
4 La salida: softmax y la pérdida −log p

Para clasificar en 10 dígitos excluyentes, la última capa entrega 10 puntajes (uno por clase) que pueden ser cualquier número. **Softmax** los convierte en probabilidades: exponencia cada puntaje (así todos quedan positivos) y divide por la suma. Resultado: números entre 0 y 1 que suman 1. Para hacerlo estable se resta el máximo c de los puntajes antes de exponenciar; las probabilidades no cambian. Después, la **pérdida de una imagen** es −log de la probabilidad que se asignó a la clase **real** (no a la más probable). La pérdida del conjunto es el promedio.

Ejemplo resuelto · puntajes s = (2, 1, 0); la clase real es la tercera

Paso	Cálculo	Resultado
Exponenciales	e^2, e^1, e^0	7.389; 2.718; 1.000
Suma	7.389 + 2.718 + 1.000	11.107
Probabilidades	cada exponencial ÷ suma	0.665; 0.245; 0.090
Comprobar	suma de las tres	1.0
Pérdida (real = 3.ª)	−log(0.090)	2.408
Con el truco	restar 2: (0, −1, −2)	mismas probabilidades

Lectura: el modelo apostó 9% a la clase real y por eso la pérdida es alta (2.408); si hubiera sido la primera clase, habría sido 0.408. **Error común**: calcular la pérdida con la probabilidad máxima en vez de la de la clase real.



La elección de salida y pérdida depende de la tarea: **regresión** (un número; error cuadrático), **clasificación binaria** (una sigmoide; entropía cruzada binaria) y **clases excluyentes** (softmax; entropía cruzada multiclase). Una función ReLU no sirve como salida de probabilidades porque no está acotada entre 0 y 1.

Lo que debes recordar

- softmax: $p_k = e^{s_k} \div \Sigma\, e^{s_j}$; suma 1
- pérdida = −log p(clase real); promedio = log loss
- La salida y la pérdida deben corresponder a la tarea

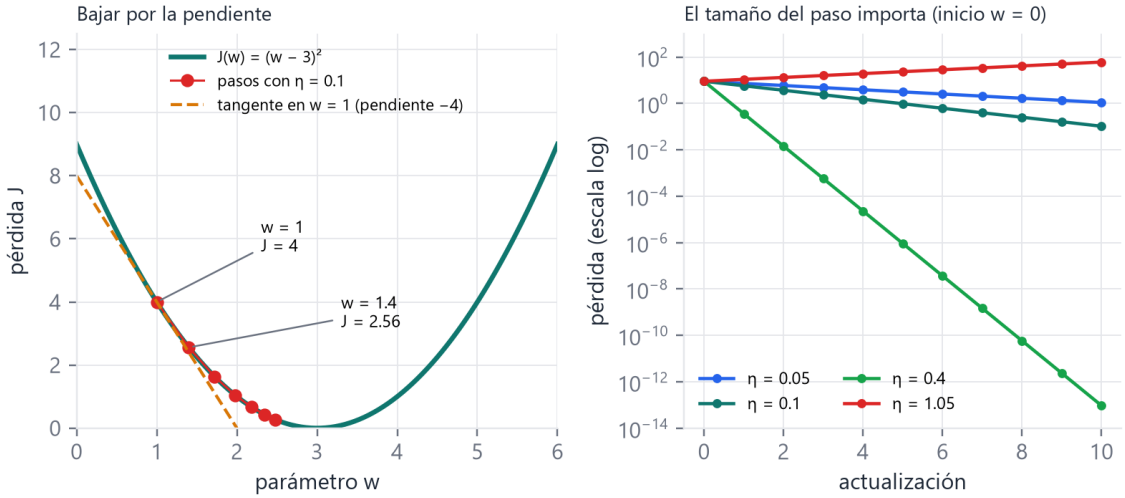
5 Aprender: minimizar J con descenso de gradiente

Aprender significa encontrar los pesos y sesgos θ que hacen pequeña una función objetivo J(θ). En la clase, J es la **pérdida media** más una **penalización** por pesos grandes (λ/2 por la suma de los pesos al cuadrado). Como J depende de miles de parámetros, se usa el **gradiente**: el vector con la pendiente de J respecto de cada parámetro. Si la pendiente de un parámetro es negativa, aumentar ese parámetro baja J; si es positiva, hay que disminuirlo. La regla es $\mathbf{w^+ = w - \eta \cdot J'(w)}$, donde η es la **tasa de aprendizaje**: cuánto se avanza en cada paso.

Ejemplo resuelto · $J(w) = (w - 5)^2$, $\eta = 0.2$, empezando en $w = 0$

Paso	w	J(w)	J′(w)	nuevo w	J(nuevo w)
1	0	25	2(0 − 5) = -10	0 − 0.2(-10) = 2	9
2	2	9	2(2 − 5) = -6	2 − 0.2(-6) = 3.2	3.24
3	3.2	3.24	2(3.2 − 5) = -3.6	3.2 − 0.2(-3.6) = 3.92	1.1664
4	3.92	1.1664	2(3.92 − 5) = -2.16	3.92 − 0.2(-2.16) = 4.352	0.4199

Lectura: en cada paso el error $|w - 5|$ se multiplica por $1 - 2\eta = 0.6$, y J por 0.36: baja de forma regular. Con $\eta = 0.5$ llegaría de golpe al mínimo y con $\eta > 1$ se alejaría (J crece). **Error común**: creer que un η más grande siempre acelera: pasado cierto punto, empeora.



La penalización mira la **magnitud** de los pesos: como $(-w)^2 = w^2$ y $(2w)^2 = 4w^2$, el signo no importa y duplicar un peso cuadruplica su aporte. El parámetro que controla cuánto pesa esa preferencia frente al ajuste a los datos se llama λ en la diapositiva y **alpha** en scikit-learn (el código de scikit-learn 1.9.1 suma α/(2·tamaño del lote) por la suma de pesos al cuadrado, es decir, $\lambda \approx \alpha \div$ lote). Alpha no es la tasa de aprendizaje.

Lo que debes recordar

- Gradiente = pendiente de J respecto de cada parámetro
- $w^+ = w - \eta \cdot J'(w)$: η es el tamaño del paso
- J = pérdida media + penalización de pesos (alpha); alpha ≠ η

6 Retropropagación: calcular todos los gradientes con la regla de la cadena

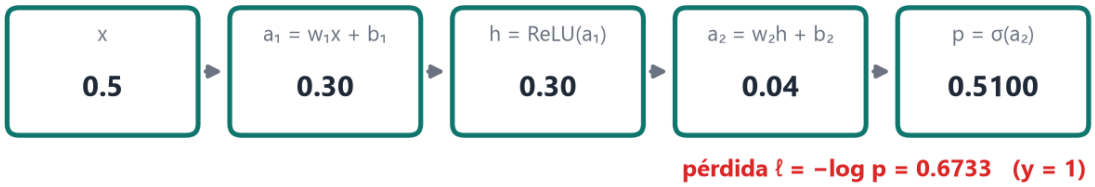
La pérdida depende de la probabilidad p, esta de a_2, esta de h, esta de a_1 y esta de los pesos: es una **cadena** de funciones. La **regla de la cadena** dice que la derivada de una cadena es el producto de las derivadas de cada eslabón. La **retropropagación** aprovecha eso: primero calcula hacia adelante (propagación) y luego recorre la cadena hacia atrás, reutilizando cada δ para el eslabón anterior. Con sigmoide y entropía cruzada binaria, $\delta_2 = p - y$. Para la capa oculta, $\delta_1 = \delta_2 \cdot w_2 \cdot \text{ReLU}'(a_1)$, donde ReLU′ vale 1 si $a_1 > 0$ y 0 si no. Y el gradiente de cada peso es el δ de su unidad por la entrada de ese peso.

Ejemplo resuelto · $x = 1$, $y = 0$ y $\theta = (w_1, b_1, w_2, b_2) = (0.6, -0.1, 0.8, -0.3)$

Paso	Cálculo	Resultado
a_1, h	$0.6 \cdot 1 - 0.1$; ReLU	0.50; 0.50
a_2, p	$0.8 \cdot 0.50 - 0.3$; $\sigma(a_2)$	0.10; 0.5250
Pérdida ($y = 0$)	$-\log(1 - 0.5250)$	0.7444
$\delta_2 = p - y$	$0.5250 - 0$	0.5250
$\partial\ell/\partial w_2$; $\partial\ell/\partial b_2$	$\delta_2 \cdot h$; δ_2	0.2625; 0.5250
δ_1	$0.5250 \cdot 0.8 \cdot 1$	0.4200
$\partial\ell/\partial w_1$; $\partial\ell/\partial b_1$	$\delta_1 \cdot x$; δ_1	0.4200; 0.4200
Actualizar ($\eta = 0.1$)	$\theta - 0.1 \cdot$gradiente	(0.5580; -0.1420; 0.7738; -0.3525)
Nueva pérdida	$p^+ = 0.4923$	0.6780 (baja)

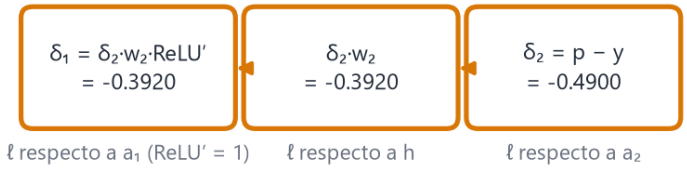
Lectura: como $y = 0$, se quiere bajar p; los cuatro gradientes son positivos, así que restarlos reduce los cuatro parámetros y la pérdida baja de 0.7444 a 0.6780. **Error común**: olvidar la entrada (x o h) al calcular el gradiente de un peso; el del sesgo, en cambio, es solo δ.

Hacia adelante (propagación): se calculan los valores



pérdida $\ell = -\log p = 0.6733$ $(y = 1)$

Hacia atrás (retropropagación): se calcula cuánto influye cada cosa en ℓ



Sensibilidades de los cuatro parámetros ($\theta = w_1, b_1, w_2, b_2$):

$\partial\ell/\partial w_1 = \delta_1 \cdot x = -0.1960$ · $\partial\ell/\partial b_1 = \delta_1 = -0.3920$ · $\partial\ell/\partial w_2 = \delta_2 \cdot h = -0.1470$ · $\partial\ell/\partial b_2 = \delta_2 = -0.4900$

Lo que debes recordar

- Primero hacia adelante (valores), luego hacia atrás (sensibilidades δ)
- $\delta_2 = p - y$; $\delta_1 = \delta_2 \cdot w_2 \cdot \text{ReLU}'(a_1)$; gradiente del peso = $\delta \times$ entrada
- Backpropagation calcula; el optimizador actualiza

7 Lotes, épocas, optimizador Adam e inicialización

Calcular el gradiente con todas las imágenes en cada paso es caro; hacerlo con una sola imagen es ruidoso. El punto medio son los **lotes**. Un **lote** es un grupo de imágenes; una **actualización** cambia los parámetros usando un lote; una **época** es una pasada por todas las imágenes de entrenamiento. Si el número de imágenes no es múltiplo del lote, el último lote es más pequeño.

Ejemplo resuelto · 1 500 imágenes de entrenamiento, lote 32, 30 épocas

Concepto	Cálculo	Resultado
Lotes completos	1500 ÷ 32	46 lotes (sobran 28)
Lote final	1500 − 46·32	28 imágenes
Actualizaciones por época	46 + 1	47
Actualizaciones en 30 épocas	30 × 47	1410

Lectura: con 1 077 imágenes y lote 64 de la clase son 16 lotes de 64 más uno de 53, es decir, 17 actualizaciones por época y 2 040 en 120 épocas. **Error común**: confundir lote con época: un lote es una parte; la época es una pasada completa.

Adam es un optimizador: la regla que usa los gradientes para decidir el cambio de cada parámetro. En vez de restar η por el gradiente, lleva dos promedios móviles: m (el gradiente reciente, la «inercia») y v (el cuadrado del gradiente, su «tamaño típico»), y da el paso $\eta \cdot \hat{m} \div (\sqrt{\hat{v}} + \varepsilon)$. Resultado: cada parámetro avanza aproximadamente η por actualización, sin importar cuán grande o pequeño sea su gradiente. La clase solo nombró a Adam (tasa 0.002); este ejemplo es una ilustración propia.

Ejemplo resuelto · primer paso de Adam desde $w = 0$, $\eta = 0.1$, con $J = (w-3)^2$ y con $J = 100(w-3)^2$

Cantidad	$J = (w-3)^2$	$J = 100(w-3)^2$
Gradiente g	-6	-600
$\hat{m}$ (inercia corregida)	-6	-600
$\sqrt{\hat{v}}$ (tamaño típico)	6	600
Paso de Adam = $\eta \cdot \hat{m}/\sqrt{\hat{v}}$	0.100	0.100
Paso del descenso simple = $\eta \cdot g$	0.6	60.0

Lectura: Adam da el mismo paso (≈ 0.1) en ambos casos; el descenso simple pasa de 0.6 a 60, y se desborda en la segunda función (figura de la guía). **Error común**: pensar que Adam siempre converge más rápido: en la función benigna avanza más lento que el descenso simple.

Inicialización. Hay que partir de algún valor. Si todos los pesos de varias unidades son iguales, esas unidades calculan lo mismo y reciben el mismo gradiente: nunca se diferencian. Por ejemplo, con tres unidades de pesos 0.5 y $x = (1, 2)$, las tres activaciones valen 1.5. Por eso se inicializa con valores distintos (la semilla fija la inicialización y el orden de las imágenes). Los pesos iniciales malos pueden hacer que la red tarde más o termine en un mínimo local; el descenso de gradiente es una búsqueda local, y no garantiza un óptimo global.

Lo que debes recordar

- Lote → actualización; época = pasada completa; el último lote puede ser menor
- Adam ≈ paso de tamaño η por parámetro, adaptado a la escala del gradiente
- Pesos iniciales iguales → unidades idénticas; la semilla fija la inicialización

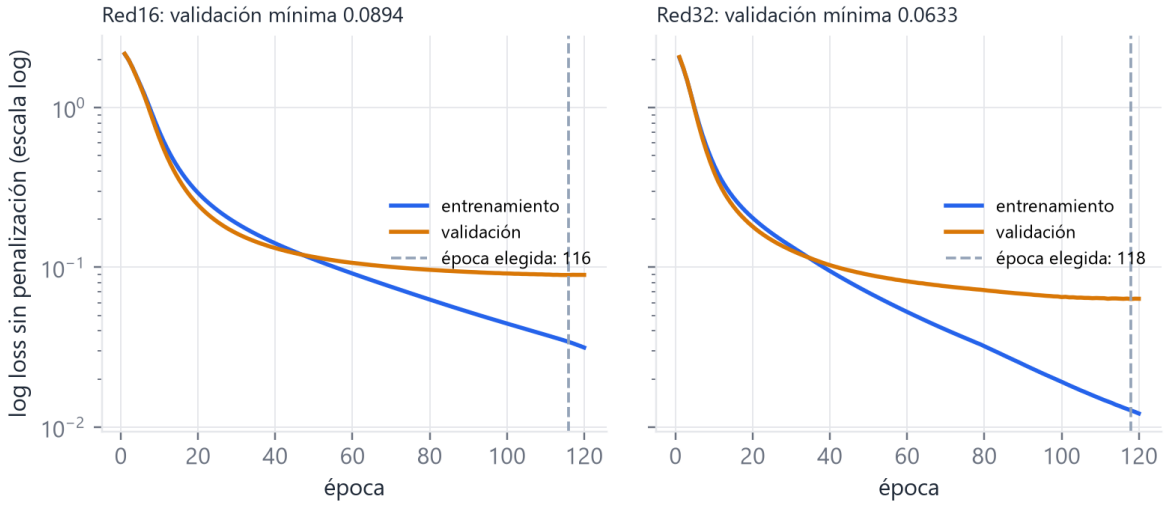
8 Leer curvas, elegir el ancho y ajustar alpha

En cada época se mide el log loss **sin penalización** en entrenamiento y en validación. Normalmente ambas bajan al comienzo; después la de entrenamiento sigue bajando mientras la de validación se aplana (y puede subir). La red se entrena 120 épocas y se **conserva la época con menor pérdida de validación** (se guarda una copia). Con dos anchos candidatos (16 y 32), se comparan sus mínimos de validación y se elige el menor, sin tocar la prueba ni reajustar con entrenamiento + validación.

Ejemplo resuelto · las curvas de la corrida del profesor (log loss)

Época	Red16 · entr.	Red16 · valid.	Red32 · entr.	Red32 · valid.
1	2.161	2.168	2.048	2.068
20	0.292	0.245	0.203	0.180
120	0.0314	0.0896	0.0122	0.0635
Mínimo de validación		0.0894 (época 116)		0.0633 (época 118)

Lectura: en la época 120 el entrenamiento de Red32 llega a 0.0122 y la validación se queda en 0.0635: la brecha crece, pero la validación sigue bajando lentamente hasta la época 118. Se elige Red32 por su menor mínimo (0.0633 frente a 0.0894). **Error común**: afirmar convergencia cuando el mínimo está justo al final del presupuesto de épocas.



El efecto de alpha (nuestra reproducción con la red 64–32–10, no es de las diapositivas): una penalización moderada reduce los pesos y mantiene o mejora la validación; una excesiva impide que la red aprenda.

alpha	Entr. (ép. 120)	Valid. (ép. 120)	Suma de W^2
0	0.0138	0.0650	349
0.0001	0.0122	0.0635	341
0.01	0.0154	0.0610	266
0.1	0.0510	0.0817	151
1	0.2916	0.2660	58
3	0.6661	0.6179	32

Con el ancho ocurre algo parecido: en nuestra reproducción, con 2 unidades la red aprende muy poco (validación 0.965), con 8 llega a 0.117, con 32 a 0.063 y con 64 vuelve a subir un poco (0.067). Elegir el ancho es un compromiso entre capacidad y ajuste, y se decide con validación.

Lo que debes recordar

- Elegir con validación (menor log loss sin penalización); la prueba, al final
- Una brecha creciente entrenamiento–validación indica sobreajuste
- Alpha moderado ayuda; demasiado impide aprender

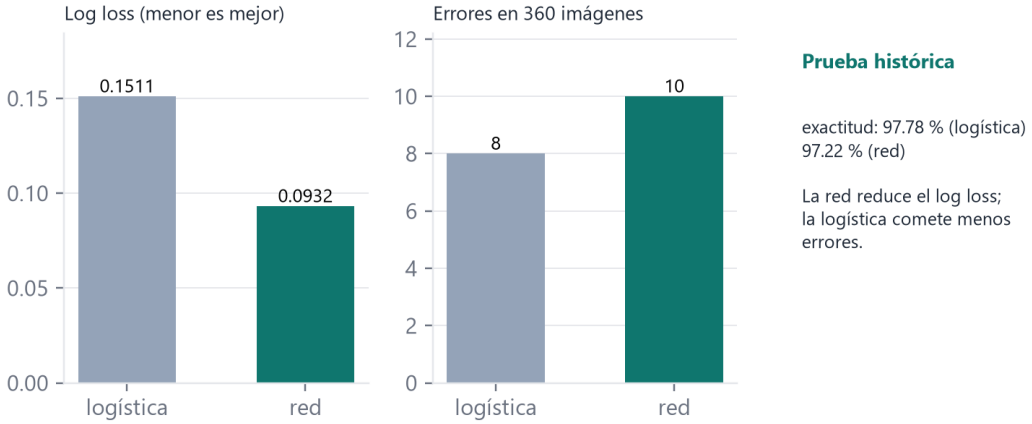
9 Evaluar: log loss, errores, confusión y alcance

Después de elegir la receta, se evalúa con la prueba y se compara con una referencia (la regresión logística). En la clase, la red tuvo log loss 0.0932 y la logística 0.1511, pero la red cometió 10 errores y la logística 8. La **exactitud** cuenta aciertos con el umbral que decide la clase; el **log loss** evalúa qué probabilidad se dio a la clase real. Un modelo puede equivocarse más veces pero con menos seguridad en sus errores y perder menos.

Ejemplo resuelto · dos modelos en dos casos con y = 1 (umbral 0.5)

Modelo	Probabilidades de la clase 1	Errores	Cálculo de la pérdida	Pérdida
A	0.60 y 0.60	0	(−log 0.60 − log 0.60) ÷ 2	0.5108
B	0.95 y 0.45	1	(−log 0.95 − log 0.45) ÷ 2	0.4249

Lectura: A no se equivoca nunca y B se equivoca una vez, pero la pérdida de B (0.4249) es menor que la de A (0.5108): B está muy seguro cuando acierta. Por eso elegir solo por exactitud o solo por log loss puede dar conclusiones distintas. **Error común**: creer que menos errores implica siempre menos pérdida.



La **matriz de confusión** detalla qué se confunde con qué: la fila es el dígito real, la columna el predicho. En la red hay 10 errores en 360 imágenes: 9→8, 1→6, 1→6, 7→8, 5→6, 6→1, 8→4, 9→3, 9→3, 3→2. Un error seguro como el 6→1 (probabilidad 0.99) pesa mucho en el log loss; por eso conviene mirar las imágenes mal clasificadas. También puede haber errores en la etiqueta del dato.

Alcance. Una semilla y una cuadrícula pequeña no establecen estabilidad; habría que repetir con varias semillas. La prueba usada ya se mostró en clase, así que la comparación es retrospectiva. Separar por imagen no demuestra desempeño con escritores nuevos. Una red densa no es invariante a desplazamientos: correr la imagen una columna cambia por completo las coordenadas que recibe. Y la receta correcta (diapositiva 46) es: separar por la unidad relevante antes de estimar transformaciones, ajustar con entrenamiento, elegir receta y época con validación, y evaluar lo fijado.

Lo que debes recordar

- Log loss (probabilidades) y exactitud (clase elegida) pueden discrepar
- Mirar la matriz de confusión y los errores seguros
- La prueba reutilizada es retrospectiva; la evaluación debe ajustarse al uso real

10 Glosario rápido

Término	Qué es
Neurona / unidad	Calcula $a = \Sigma wx + b$ y entrega $h = \varphi(a)$.
Peso, sesgo	Parámetros que se aprenden; el sesgo desplaza la suma.
Activación	El resultado h de una unidad para una entrada. No es una probabilidad.
Capa oculta, ancho, profundidad	Capas intermedias; unidades por capa; número de capas.
Hiperparámetro	Se fija antes de entrenar: ancho, capas, η, lote, alpha, épocas.
Softmax	Convierte puntajes en probabilidades que suman 1.
Pérdida, log loss	$-\log p$ de la clase real; promedio sobre las imágenes.
Gradiente, η	Pendiente de J por parámetro; η es el tamaño del paso.
Retropropagación	Cálculo de gradientes con la regla de la cadena.
Lote, actualización, época	Grupo de imágenes; un cambio de parámetros; una pasada completa.
Adam	Optimizador con inercia y paso adaptado a la escala del gradiente.
Alpha	Penalización L2 de los pesos en scikit-learn (no es la tasa).
Sobreajuste	El entrenamiento mejora, pero la validación no.